Listing A.9 Error Boundary component's render method selectively ignoring specific known errors

```
render() {
  if (this.state.error && !/js-agent\.newrelic\.com|reading 'pare
    return <h1>Something went wrong...</h1>;
  }
  return this.props.children;
}
```

By strategically ignoring specific error scenarios, the `render` method ensures application stability and functionality while appropriately handling other error types. A frontend candidate who thinks critically about these trade-offs demonstrates superior skill compared to peers. Error scenarios, logs, and application stability intersect with critical concepts:

- Logging = observability (vital for infrastructure and DevOps teams)
- Application stability = resiliency (key for business and product teams)

When failures inevitably occur, the UI's role in broader auto-scaling and RTO (Recovery Time Objective) strategies becomes crucial. Exploring these topics during interviews reveals a candidate's depth of understanding and alignment with modern engineering practices.

Listing A.10 demonstrates the `render()` method of an Error Boundary that addresses a "Loading chunk [\d]+ failed" error, often associated with code-splitting issues. Upon encountering this specific error, the component will trigger a page reload after a 5-minute delay.

Listing A.10 Error Boundary component's render method reloading the page after 5 minutes to recover from chunk loading errors

```
render() {
  if (this.state.error) {
    if (/Loading chunk [\d]+ failed/.test(this.state.error)) { #A
      console.error(`reload to recover from the error: ${this.sta
      const lastReloadTime = localStorage.getItem('reload-to-reco
      if (!lastReloadTime || Date.now() - Number(lastReloadTime)
        localStorage.setItem('reload-to-recover-time', String(Dat
        window.location.reload();
        return;
      }
```